RootCause: AI-powered diagnostics assistant

I. Overview

We propose **RootCause**, an on-demand diagnostic agent powered by a fine-tuned Small Language Model (SLM) that combines Retrieval-Augmented Generation (RAG) with active tool execution via the Model Context Protocol (MCP).

How It Works

Rather than operating as a continuous background daemon, RootCause follows a human-in-the-loop, request-driven workflow:

1. **User Input:** The developer submits a custom input (an error log, exception trace, or behavioral anomaly).
2. **Contextual Retrieval (RAG):** The agent queries an ingested vector store of internal runbooks, architecture docs, and past incident post-mortems to find similar historical failures.
3. **Active Investigation (MCP Tools):** Before guessing an answer, the agent reasons over the input and triggers deterministic tools exposed via a standalone MCP server (e.g., parsing log structures, validating configuration parameters, or querying simulated service statuses) to confirm its hypothesis.
4. **Grounded Output:** The agent synthesizes the retrieved documentation and tool outputs into a structured diagnosis and actionable fix, filtered through strict guardrails to block hallucinated commands or unsafe operations.

II. Motivation

When backend services fail or throw obscure exceptions, software engineers and DevOps teams face significant cognitive overload. Diagnosing a single failure typically requires manually cross-referencing cryptic stack traces against fragmented internal documentation, historical post-mortem reports, API specifications, and system state metrics.

This manual triage process introduces two critical bottlenecks:

- **High Mean Time to Resolution (MTTR):** Engineers spend more time hunting for context across disparate knowledge bases than actually fixing the underlying bug.
- **Context Fragmentation:** Institutional knowledge from past incidents is often locked in unstructured markdown files or tickets, leading to repeated investigation of previously solved errors.

Currently, teams rely on three primary paradigms to assist with incident troubleshooting, each carrying distinct trade-offs:

Approach / Solution	Pros	Cons
1. General-Purpose Commercial LLMs *(e.g., pasting logs into ChatGPT or Claude)*	• Zero setup friction. • Strong general coding knowledge and syntax familiarity.	• **Zero internal context:** Blind to proprietary architecture and internal runbooks. • **High hallucination risk:** Invents non-existent CLI flags or generic fixes. • **Privacy concerns:** Pasting sensitive production logs into third-party APIs violates enterprise compliance.
2. Enterprise AIOps Platforms *(e.g., Datadog Bits AI, New Relic AI)*	• Deep integration with live telemetry. • Automated anomaly correlation.	• **Heavyweight & intrusive:** Requires continuous, real-time agent instrumentation across all servers. • **Prohibitive cost:** Vendor lock-in and expensive per-host licensing.
3. Naive RAG Chatbots *(Static vector search over docs)*	• Grounds responses in internal documentation. • Easy to prototype.	• **Passive retrieval only:** Cannot actively execute diagnostic tools (e.g., checking a status code or querying a config rule) to verify a hypothesis before answering.

III. Goals

Our goal is to solve this diagnostic bottleneck by building an on-demand, user-triggered AI assistant. Instead of requiring complex, always-on background monitoring infrastructure, a developer simply pastes an error log, stack trace, or symptom description into the application, and the system autonomously gathers the necessary context to output a grounded root-cause diagnosis and remediation plan.

What Makes Our Solution Different and Better

- **On-Demand Simplicity vs. Heavyweight AIOps:** By triggering strictly on user input rather than streaming real-time telemetry, RootCause eliminates continuous infrastructure overhead while still delivering deep diagnostic reasoning when an engineer needs it.
- **Active Verification via MCP vs. Passive RAG:** Standard RAG systems only summarize text. By decoupling tools into an MCP server, our agent actively tests assumptions and fetches deterministic metadata during its reasoning loop, drastically reducing false positives.
- **Fine-Tuned SLM vs. Commercial APIs:** While we will benchmark our initial pipeline against commercial models, our final architecture centers on a task-specific fine-tuned SLM (e.g., Llama-3-8B or Phi-3). Fine-tuning an SLM on a curated Golden Dataset of backend failures ensures reliable JSON tool-calling, strict adherence to output schemas, low latency, and complete data privacy—achieving enterprise-grade diagnostics without external API dependency.